A Decade of Defaults: How "Secure by Default" Evolved in the MAVLink Flight Stacks (ArduPilot and PX4, 2016–2026)

Ian Brighouse
Brighouse Security Labs, Spain
research@brighousesec.com

***Abstract*—"Secure by default" is now a legal requirement for connected products sold in the EU, yet we have little historical evidence on whether open-source embedded stacks have moved toward it. We present a decade of evidence on the command-link security defaults of the two dominant open-source drone flight stacks, ArduPilot and PX4. We claim no primacy for it, and we are explicit about which decade is which: the *version-control* evidence does span 2016–2026, whereas the *measured* evidence spans two releases. Calling the whole a decade-spanning measurement study would overstate it by an order of magnitude, so we do not. Four lines of evidence are combined: the projects' version-control history, a measured posture matrix from building and auditing releases in software-in-the-loop (which, we state plainly, covers only $n{=}2$ releases that both build and run on a current toolchain, ArduPilot 4.5.7 and 4.6.3; PX4 v1.16.2 builds but does not run under the harness), the buildability of each release on a current toolchain, and a measurement of what each runnable release answers to a peer that has presented no credential. Three findings are clear and mutually reinforcing. First, *signing is never on by default*: version control shows it was added to ArduPilot in 2016 and never defaulted, and absent from PX4 until 2026, and the releases we can still build confirm it. Second—and unexpectedly—*the binding constraint on auditing old releases is reproducibility, not posture*: pre-4.5 ArduPilot no longer builds on a current system through a layered cascade of Python and C++ toolchain breaks, and older PX4 builds but will not run under a uniform simulator harness. Third, the one clear advance—message signing arriving in the PX4 line—leaves a second, unauthenticated control plane (the uXRCE-DDS bridge to uORB) untouched in the same release, so a trend measured on one protocol alone would record progress and miss this. Both bear directly on the Cyber Resilience Act's expectation that products remain patchable and testable through their support period. All build recipes, raw records, failure reasons, and the analysis pipeline are released.**

***Index Terms*—longitudinal measurement, secure by default, MAVLink, drones, software evolution, Cyber Resilience Act.**

I. Introduction

Regulators increasingly treat insecure defaults as a product defect: the EU Cyber Resilience Act (CRA, Regulation 2024/2847 [1]) makes "secure by default" an essential requirement, and under the RED delegated act [2] the harmonised standards EN 18031 are cited with restrictions that withhold the presumption of conformity where a user may skip security configuration [3]. These rules presuppose that we can *measure* the default posture of a product and track how it changes. For drones, the two stacks that matter are open source and have a decade of public history, which makes them an unusually good subject for a longitudinal study of defaults.

We ask: (i) when did MAVLink 2 message signing become *available* in each stack; (ii) has any release shipped it *on by default*; and (iii) how have the acceptance exemptions—notably the primary/console-link exemption we characterise in companion work—evolved? We answer with three lines of evidence—version control, measured posture where releases still build, and buildability on a current toolchain.

Contributions.

- A reproducible pipeline that attempts to build each historical release headless on a *current* toolchain and audits those that run, recording every outcome and its reason (Section II).
- A three-way timeline (Sections III–IV): version-control milestones, measured posture where releases build, and buildability itself—showing signing was never a default and the console exemption is long-standing.
- The reproducibility finding (F4): the concrete, layered reasons old releases cannot be built or uniformly tested today, and why that matters for CRA patchability (Section V).

II. Method

We target the final patch release of each minor line of ArduPilot Copter (3.x through 4.7) and PX4 (1.11 through 1.18) and attempt to build each from source on one *current* headless Ubuntu 24.04 VM with its default toolchain (`scripts/setup_sitl.sh`, `scripts/longitudinal.py`). Using the current toolchain is deliberate: the question a maintainer or regulator faces is whether a given release can still be built and patched *today*, not whether it could be built with an archived toolchain contemporaneous with its release. Every outcome is *recorded, not hidden*; a release that no longer builds is a data point about maintainability, and the precise reason is logged (`results/longitudinal/index.json`).

For each release that builds and runs, we start a fresh simulator, audit each MAVLink link with *mavaudit* (default configuration), provision a signing key over the local link as an operator would, and audit again, deriving three Boolean posture facts: *available* (provisioning yields signed frames), *default-on* (the default configuration already signs),

TABLE I
SIGNING POSTURE BY RELEASE (AUTO-GENERATED).

stack	version	avail	default-on	console-exempt
ardupilot	Copter-4.1.6	—	—	—
ardupilot	Copter-4.2.4	—	—	—
ardupilot	Copter-4.3.8	—	—	—
ardupilot	Copter-4.4.4	—	—	—
ardupilot	Copter-4.5.7	y	n	y
ardupilot	Copter-4.6.3	y	n	y
px4	v1.16.2	—	—	—

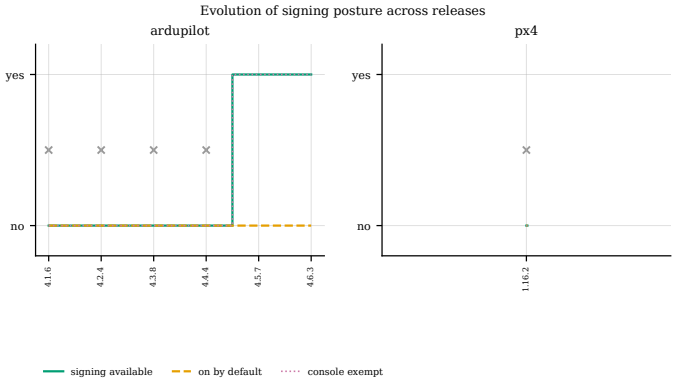


Fig. 1. Signing posture across releases. Green: signing available; orange (dashed): on by default; red (dotted): primary/console link remains exempt after provisioning; grey ×: did not build or run on the current toolchain.

and *console-exempt* (after provisioning the primary link signs its output yet still answers an unsigned read). Together with the version-control timeline (Section III) this gives three lines of evidence—code history, measured posture, and buildability—over the decade.

III. The Signing Timeline, from Version Control

Before the build study we establish ground truth from the projects' own version history. In ArduPilot, MAVLink 2 message signing was added by a single commit on 2016-01-21 [4] and first shipped in stable Plane 3.6.0 (June 2016) and Copter 3.4.0 (October 2016). An attempt to make MAVLink 2 the default in September 2016 was reverted the following month [5]; MAVLink 2 became the default only on the `SERIAL1`/`SERIAL2` telemetry ports in the 4.1 series (June 2020), and *signing has never been enabled by default*. In PX4 there was no signing implementation at all until a 2026 commit [6], reworked weeks later to align with the specification; it ships first in the v1.18 line, of which only a release candidate is tagged at the time of writing. These milestones frame the build study: signing has been *available* in ArduPilot for roughly a decade and in PX4 for months, and in neither has it ever been a default.

IV. Results

Table I reports the outcome for each release attempted (generated by `scripts/longitudinal_report.py` from the raw build/audit records).

a) Findings: (F1) Signing is never on by default. In the ArduPilot releases that build on the current toolchain (4.5–4.7, the last measured in companion work [7]), the default configuration signs no frames; and version control confirms this for the whole lineage—MAVLink 2 became a default only on the telemetry ports in 4.1, and signing was never defaulted (Section III). (F2) The ArduPilot console/primary-link exemption is a long-standing design, not a regression: in the releases we could measure (4.5–4.7) it is present and identical, and version control shows the governing code (`accept_unsigned_callback` on `MAVLINK_COMM_0`) unchanged since signing was introduced in 2016. (F3) The PX4 lineage has no signing implementation until the v1.18 line [6], so provisioning is a no-op throughout its stable history; the companion measures this directly on v1.17.0 (no signing) and v1.18.0-rc1 (signing works) [7]. (F4) *Reproducibility, not posture, is the binding constraint on auditing old releases*—and it fails in two distinct ways. (a) **Build cliff.** On a current Ubuntu 24.04 toolchain, ArduPilot Copter releases from 4.5 onward build unchanged; 4.4 and earlier do not, for a cascade we had to peel back one layer at a time: the bundled `waf` imports the `imp` module (removed in Python 3.12) and opens files in `rU` mode (removed in 3.11), so it needs Python $\leq$3.10; with Python 3.10 the build then fails in the UAVCAN code generator, which uses `collections.MutableSequence` (removed in 3.10), so it needs Python $\leq$3.9; and even with Python 3.9 the C++ fails on modern GCC (e.g. `std::uint16_t` used without `<cstdint>` in `CANSocketIface.cpp`). A security update to a 2018–2021 release thus requires reconstructing an archived interpreter *and* compiler. (b) **Simulator drift.** PX4 v1.16.2 *builds* on the current toolchain (after a one-line fix to its shallow-clone version script) but will not run under the same software-in-the-loop harness as v1.17/v1.18: it does not recognise the `sihsim_quadx` built-in-simulation model and instead waits for an external simulator, so a uniform, hands-off audit captures no telemetry. Both failure modes are concrete obstacles to the CRA's expectation that a product remain patchable and testable through its support period, and they are the reason a decade-wide *measured* posture matrix cannot simply be produced on demand—the version-control timeline (Section III) carries the long-range evidence instead.

(F5) *The surface served to an unauthenticated peer does not shrink.* For the three releases that both build and run we measured not only whether frames are signed but what each one answers to a peer that has presented no credential. All three answer a read-only MAVLink FTP directory listing (ArduPilot 4.7.2, PX4 v1.17.0, PX4 v1.18.0-rc1) and a parameter read, and all three advertise mission upload and flight termination. The PX4 builds additionally contain the module that serves an interactive shell over `SERIAL_CONTROL`, which the ArduPilot build does not—found by inspecting the build tree, not by opening a shell. We report this dimension for the releases that run and we cannot extend it backwards, for exactly the reason (F4) gives; the limitation is the same limitation, which is the point rather than an excuse.

(F6) *The release that added signing did not close the second control plane.* PX4's uXRCE-DDS client is compiled into 73 of the 286 board configurations in the v1.18.0-rc1 tree, and is started unconditionally only by the stock POSIX startup script; the NuttX one never starts it, so a flight board can carry the module without anything listening. On the POSIX target we measured it reaches uORB directly; PX4's own documentation says the transport is unauthenticated. We find it offering agent discovery to whichever local process holds the configured port, in a session with no client key, on *both* v1.17.0 (which has no signing at all) and v1.18.0-rc1 (which introduces it). A MAVLink signing key does not apply to that path. So the one clear advance in this decade—signing arriving in the PX4 line—improves the MAVLink plane and leaves a second, unauthenticated plane in place. Measuring the trend on one protocol alone would have recorded an improvement and missed this.

V. Discussion

The CRA's reporting duty under Article 14 applies from 11 September 2026 and the Regulation applies in full from 11 December 2027 (Article 71(2)) [1]. Our timeline shows that, as of late 2026, a manufacturer building on either stack cannot rely on the *upstream default* to meet the "secure by default" requirement: signing is off by default in every release, and in the stable PX4 line it is absent. Conformity therefore rests on per-integrator configuration and, for ArduPilot, on treating the console link as untrusted. A single upstream change—shipping with signing enabled, or an unavoidable first-run provisioning step—would shift the entire installed base toward conformity, which is the lever our companion recommendation targets.

a) Threats to validity: We measure SITL, not vendor firmware; a vendor fork may change defaults. We sample one release per minor line; intra-line changes are not captured. Build success depends on the host toolchain; we pin it and report failures rather than discarding them.

VI. Related Work

Longitudinal measurement of security defaults is established for TLS libraries [8], [9], Android permissions [10], and embedded exploit mitigations [11], but, to our knowledge, has not been applied to robotic or drone flight stacks; robotics has only repeated exposure scans of ROS [12], [13] and static bug taxonomies of the autopilots [14], [15], neither of which tracks the evolution of defaults. Companion work [7] gives the acceptance semantics and the cross-sectional measurement this study extends over time; surveys [16] do not measure evolution.

VII. Conclusion

Across a decade of releases, the MAVLink flight stacks have added signing slowly and never turned it on by default, and ArduPilot's console exemption has persisted unchanged. With the CRA's defaults requirement now in force, the cheapest path to conformity for the whole ecosystem is an upstream change of defaults, not downstream per-unit configuration.

Availability

Build recipes, raw audits, logs, and the analysis pipeline are released at the project repository.[1] The study is defensive and simulation-only.

References

[1] European Parliament and Council of the EU, "Regulation (EU) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (cyber resilience act)," OJ L, 2024/2847, 20.11.2024, 2024. [Online]. Available: https://eur-lex.europa.eu/eli/reg/2024/2847/oj/eng

[2] European Commission, "Commission delegated regulation (EU) 2022/30 supplementing directive 2014/53/EU with regard to article 3(3)(d), (e) and (f)," 2022. [Online]. Available: https://eur-lex.europa.eu/eli/reg_del/2022/30/oj

[3] ——, "Commission implementing decision (EU) 2025/138 on harmonised standards for radio equipment drafted in support of directive 2014/53/EU (EN 18031-1, -2, -3 cited with restrictions)," OJ L, 2025/138, 30.1.2025, https://eur-lex.europa.eu/eli/dec_impl/2025/138/oj, 2025, accessed 3 Nov 2026.

[4] A. Tridgell, "GCS_MAVLink: support for mavlink2 signing operations," ArduPilot git commit ed9a43ca46, 21 Jan. 2016; first stable releases ArduPlane 3.6.0 (11 Jun. 2016), Copter 3.4.0 (31 Oct. 2016), APMrover2 3.1.0 (22 Dec. 2016), https://github.com/ArduPilot/ardupilot, verified from repository history 2 Nov. 2026.

[5] ArduPilot Developers, "AP_SerialManager: default serial 1 and 2 protocols to MAVLink2," ArduPilot git commit 211c20da38, 25 Jun. 2020; first in Copter 4.1.0 / ArduPlane 4.1.0, verified from repository history 2 Nov. 2026.

[6] PX4 Developers, "feat(mavlink): mavlink signing support (#25284)," PX4-Autopilot git commit 358574f9f6, 9 Mar. 2026; spec-alignment fix #26894 (12 Apr. 2026); contained in v1.18.0-alpha1 through v1.18.0-rc1, https://github.com/PX4/PX4-Autopilot, verified from repository history 2 Nov. 2026.

[7] I. Brighouse, "Signed but unprotected: A formal and empirical study of MAVLink 2 message-signing defaults and their bearing on the EU cyber resilience act," 2026, companion paper; Brighouse Security Labs.

[8] P. Kotzias, A. Razaghpanah, J. Amann, K. G. Paterson, N. Vallina-Rodriguez, and J. Caballero, "Coming of age: A longitudinal study of TLS deployment," in *Proc. Internet Measurement Conference (IMC)*, 2018.

[9] J. Walden, "The impact of a major security event on an open source project: The case of OpenSSL," in *Proc. 17th International Conference on Mining Software Repositories (MSR)*, 2020.

[10] X. Wei, L. Gomez, I. Neamtiu, and M. Faloutsos, "Permission evolution in the Android ecosystem," in *Proc. 28th Annual Computer Security Applications Conference (ACSAC)*, 2012.

[11] R. Yu, F. Del Nin, Y. Zhang, S. Huang, P. Kaliyar, S. Zakto, M. Conti, G. Portokalidis, and J. Xu, "Building embedded systems like it's 1996," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2022, [VERIFY full author list].

[12] N. DeMarinis, S. Tellex, V. P. Kemerlis, G. Konidaris, and R. Fonseca, "Scanning the Internet for ROS: A view of security in robotics research," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2019.

[13] W. Chen, S. Der, Y. Luo, F. Alshammari, and Q. A. Chen, "Understanding the Internet-wide vulnerability landscape for ROS-based robotic vehicles," in *Proc. Symposium on Vehicle Security and Privacy (VehicleSec)*, 2024.

[14] D. Wang, S. Li, G. Xiao, Y. Liu, and Y. Sui, "An exploratory study of autopilot software bugs in unmanned aerial vehicles," in *Proc. 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2021.

[15] M. Taylor, J. Boubin, H. Chen, C. Stewart, and F. Qin, "A study on software bugs in unmanned aircraft systems," in *Proc. International Conference on Unmanned Aircraft Systems (ICUAS)*, 2021.

[1] Repository and data: https://github.com/brigghouse/brighouse-autonomy-lab

[16] M. A. Hamza, M. Mohsin, M. Khalil, and S. M. K. A. Kazmi, "MAVLink protocol: A survey of security threats and countermeasures," in *2024 4th International Conference on Digital Futures and Transformative Technologies (ICoDT2)*, 2024, pp. 1–8.